



มหาวิทยาลัยขอนแก่น

วิทยา จวิทยา ปัญญา

KHON KAEN UNIVERSITY



Normalization (Part 1)

Assoc. Prof. Dr. Kanda Runapongsa Saikaew
Faculty of Engineering
Khon Kaen University



คณะวิศวกรรมศาสตร์ มหาวิทยาลัยขอนแก่น
FACULTY OF ENGINEERING KHON KAEN UNIVERSITY

Agenda

- Relational Design Overview
- Lossless and Lossy Decomposition
- Normalization Theory
- Functional Dependencies



Terminologies in Relational Database Design

- Database: a structured collection of data organized for efficient retrieval manipulation
- Schema: The logical structure that defines how data is organized in a database
- Redundancy: Unnecessary repetition of data in a database
- Anomaly: An inconsistency or problem that can occur when updating, inserting, or deleting data
- Normalization: The process of organizing data to minimize redundancy and dependency



Designing a Database Schema

- Designing a database schema
 - Usually many designs possible
 - Some are (much) better than others
 - How do we choose?
- Often use higher-level design tools, but ...
 - Some designers go straight to relations
 - Useful to understand why tools produce certain schemas
- Very nice theory for relational database design



Is This a Good Design?

```
[mysql> describe books;
```

Field	Type	Null	Key	Default	Extra
BookID	int	NO	PRI	NULL	
Title	varchar(100)	YES		NULL	
Author	varchar(100)	YES		NULL	
BorrowerName	varchar(100)	YES		NULL	
BorrowerAddress	varchar(200)	YES		NULL	
BorrowDate	date	YES		NULL	

```
6 rows in set (0.00 sec)
```

```
[mysql> select * from books;
```

BookID	Title	Author	BorrowerName	BorrowerAddress	BorrowDate
1	To Kill a Mockingbird	Harper Lee	John Smith	789 Oak St, Newtown, USA	2023-06-01
2	1984	George Orwell	John Smith	789 Oak St, Newtown, USA	2023-06-02
4	The Great Gatsby	F. Scott Fitzgerald	NULL	NULL	NULL
5	To Kill a Mockingbird	Harper Lee	Sarah Johnson	456 Elm St, Othertown, USA	2023-06-05

```
4 rows in set (0.00 sec)
```



Redundancy Problem

- Redundancy: BorrowerName and BorrowerAddress are repeated
- **SELECT** * **FROM** Books **WHERE** BorrowerName = 'John Smith';

```
[mysql> SELECT * FROM Books WHERE BorrowerName = 'John Smith';
```

BookID	Title	Author	BorrowerName	BorrowerAddress	BorrowDate
1	To Kill a Mockingbird	Harper Lee	John Smith	789 Oak St, Newtown, USA	2023-06-01
2	1984	George Orwell	John Smith	789 Oak St, Newtown, USA	2023-06-02

```
2 rows in set (0.00 sec)
```



Update Anomaly

- Changing a borrower's address requires multiple updates
- **UPDATE** Books **SET** BorrowerAddress = '789 Oak St, Newtown, USA' **WHERE** BorrowerName = 'John Smith';

```
[mysql> SELECT * FROM Books WHERE BorrowerName = 'John Smith';
```

BookID	Title	Author	BorrowerName	BorrowerAddress	BorrowDate
1	To Kill a Mockingbird	Harper Lee	John Smith	789 Oak St, Newtown, USA	2023-06-01
2	1984	George Orwell	John Smith	789 Oak St, Newtown, USA	2023-06-02

```
2 rows in set (0.00 sec)
```

```
[mysql> UPDATE Books SET BorrowerAddress = '789 Oak St, Newtown, USA' WHERE BorrowerName = 'John Smith';
```

```
Query OK, 0 rows affected (0.00 sec)
```

```
Rows matched: 2 Changed: 0 Warnings: 0
```



Insertion Anomaly

- Insertion Anomaly: We can insert data without a book title and a book author
- **INSERT INTO** Books (BookID, BorrowerName, BorrowerAddress) **VALUES** (6, 'Emma Watson', '321 Pine St, Hollywood, USA');

```
[mysql> INSERT INTO Books (BookID, BorrowerName, BorrowerAddress) VALUES (6, 'Emma Watson', '321 Pine St, Hollywood, USA');  
Query OK, 1 row affected (0.01 sec)
```

```
mysql> select * from books;
```

BookID	Title	Author	BorrowerName	BorrowerAddress	BorrowDate
1	To Kill a Mockingbird	Harper Lee	John Smith	789 Oak St, Newtown, USA	2023-06-01
2	1984	George Orwell	John Smith	789 Oak St, Newtown, USA	2023-06-02
4	The Great Gatsby	F. Scott Fitzgerald	NULL	NULL	NULL
5	To Kill a Mockingbird	Harper Lee	Sarah Johnson	456 Elm St, Othertown, USA	2023-06-05
6	NULL	NULL	Emma Watson	321 Pine St, Hollywood, USA	NULL

```
5 rows in set (0.00 sec)
```



Deletion Anomaly

- Removing a book also removes borrower information
- **DELETE FROM Books WHERE Title = 'To kill a Mockingbird';**

```
[mysql> select * from books;
```

BookID	Title	Author	BorrowerName	BorrowerAddress	BorrowDate
1	To Kill a Mockingbird	Harper Lee	John Smith	789 Oak St, Newtown, USA	2023-06-01
2	1984	George Orwell	John Smith	789 Oak St, Newtown, USA	2023-06-02
4	The Great Gatsby	F. Scott Fitzgerald	NULL	NULL	NULL
5	To Kill a Mockingbird	Harper Lee	Sarah Johnson	456 Elm St, Othertown, USA	2023-06-05
6	NULL	NULL	Emma Watson	321 Pine St, Hollywood, USA	NULL

```
5 rows in set (0.00 sec)
```

```
[mysql> DELETE FROM Books WHERE Title = 'To kill a Mockingbird';
```

```
Query OK, 2 rows affected (0.01 sec)
```

```
[mysql> select * from books;
```

BookID	Title	Author	BorrowerName	BorrowerAddress	BorrowDate
2	1984	George Orwell	John Smith	789 Oak St, Newtown, USA	2023-06-02
4	The Great Gatsby	F. Scott Fitzgerald	NULL	NULL	NULL
6	NULL	NULL	Emma Watson	321 Pine St, Hollywood, USA	NULL

```
3 rows in set (0.00 sec)
```



Summary about Table Books Design

- This design is problematic because:
- It has **redundancy**: If a borrower takes multiple books, their name and address are repeated.
- It can lead to **update anomalies**: If a borrower's address changes, it needs to be updated in multiple places.
- It has **insertion anomalies**: We can insert a book without book title and book author
- It has **deletion anomalies**: If we delete a book record, we lose borrower information.



Better books Design

- Books(BookID, Title, Author)
- Borrowers(BorrowerID, BorrowerName, BorrowerAddress)
- Loans(LoanID, BookID, BorrowerID, BorrowDate)
- **Eliminating redundancy**: Borrower information is stored only once.
- **Avoiding update anomalies**: A borrower's address is updated in one place.
- **Allowing for new borrowers or books to be added independently.**
- **Preserving borrower information** even if all books are returned.



Good Relational Database Design

- Good relational database design refers to the process of organizing data in a relational database management system to minimize redundancy, ensure data integrity, and provide efficient access to information.
- It involves structuring the database schema in a way that accurately represents the relationships between different entities while adhering to normalization principles.



Key Characteristics of a Good Relational Database Design (1/3)

- **Minimized Data Redundancy**

- Data is stored in one place to avoid inconsistencies and reduce storage requirements.

- **Data Integrity**

- The design enforces data accuracy and consistency across the database.

- **Efficient Data Access**

- The structure allows for quick and easy retrieval of information.



Key Characteristics of a Good Relational Database Design (2/3)

- **Flexibility**

- The design can accommodate future changes and additions with minimal restructuring

- **Normalized Structure**

- The design follows normalization rules (typically up to 3NF or BCNF) to eliminate anomalies.

- **Clear Relationships**

- Entity relationships are well-defined and properly implemented (e.g., through foreign keys).



Key Characteristics of a Good Relational Database Design (3/3)

- Scalability

- The design can handle growth in data volume and complexity.

- Security

- The structure supports implementation of access controls and data protection measures.

- A good relational database design is one that effectively balances these characteristics to meet the specific needs of the application or organization it serves, while maintaining data accuracy, consistency, and accessibility.



RDBMS Design and Functional Dependencies

- **Functional dependencies (FDs)** play a crucial role in achieving good relational database design.
- They are the foundation for understanding and implementing **normalization**, which is a key process in creating efficient and consistent database structures



What is a Functional Dependency?

- A functional dependency (FD) is a constraint between two sets of attributes in a relation from a database.
- Given a relation R , a set of attributes X in R is said to functionally determine another set of attributes Y , also in R , (written $X \rightarrow Y$) if and only if each X value in R is associated with precisely one Y value in R .
- In other words, $X \rightarrow Y$ means that the values of Y are determined by the values of X .



Key Characteristics of FD

- It's a property of the semantics or meaning of the attributes
- It must hold for all possible relation instances
- It's based on the notion that some attributes' values uniquely determine the values of certain other attributes.



Examples of FD

- In a relation

Student(StudentID, Name, DateOfBirth)

- We might have the functional dependency:

$\text{StudentID} \rightarrow \text{Name, DateOfBirth}$

- This means that for each StudentID, there is only one associated Name and DateOfBirth.



How FDs relate to Good Database Design (1/8)

1. Identifying Data Relationships

- Functional dependencies help identify the inherent relationships between attributes in a database
- A functional dependency $X \rightarrow Y$ means that the value of X determines the value of Y.
- This understanding is crucial for:
 - Determining which attributes should be grouped together in a relation
 - Identifying potential keys for relations
 - Understanding the business rules and constraints of the data



How FDs relate to Good Database Design (2/8)

- **Example of Identifying Data Relationships**

- In a poorly designed table **Students(StudentID, Name, SectionID, SectionName, Professor)**, we might identify these FDs:

- **StudentID $\rightarrow$ Name**

- **SectionID $\rightarrow$ SectionName, Professor**

- This demonstrates the need to separate section information from student information



How FDs relate to Good Database Design (3/8)

2. Minimizing Data Redundancy

- Good database design aims to minimize data redundancy, which is where functional dependencies are invaluable
- By identifying FDs, we can:
 - Recognize repeated data
 - Separate attributes into different relations based on their dependencies
- This process helps in achieving higher normal forms, reducing data duplication and potential inconsistencies.



How FDs relate to Good Database Design (4/8)

3. Ensuring Data Integrity

- Functional dependencies help maintain data integrity by:
 - Identifying the correct keys for each relation
 - Ensuring that update, insert, and delete anomalies are minimized
- When we design based on FDs, we create a structure where each piece of data is stored in only one place, reducing the risk of inconsistencies.



How FDs relate to Good Database Design (5/8)

- Problem: Consider a poorly designed Student table:
- `Students(StudentID, Name, Major, Department)`
- With functional dependencies:
 - `StudentID → Major` (each student has one major)
 - `Major → Department` (each major belongs to one department)
- What goes wrong:
 - Redundancy: "Computer Science Department" is repeated for every CS student
 - Update problems: If the CS department changes its name, you must update hundreds of student records
 - Inconsistency risk: Some CS students might show "Comp Sci Dept" while others show "Computer Science Department"



How FDs relate to Good Database Design (6/8)

- Better solution: Separate the tables:
 - Students(StudentID, Name, Major)
 - Majors(Major, Department)
- Why this works better:
 - Department name stored only once per major
 - Changing a department name requires updating just one record
 - Eliminates the transitive dependency (StudentID $\rightarrow$ Department through Major)
 - Maintains data consistency through proper normalization
- This demonstrates how functional dependencies guide us to separate related but distinct information into appropriate tables.



How FDs relate to Good Database Design (7/8)

- **Facilitating Normalization**
- Normalization uses functional dependencies to systematically improve database design through progressive steps:
 - 1NF: Eliminates repeating groups and ensures atomic values
 - FDs help by: Identifying which attributes should be in separate rows vs. columns
 - 2NF: Removes partial dependencies (non-key attributes depending on part of a composite key)
 - FDs help by: Revealing when attributes depend on only part of the primary key
 - 3NF: Eliminates transitive dependencies ($A \rightarrow B \rightarrow C$ situations)
 - FDs help by: Identifying chains of dependencies that create redundancy
 - BCNF: Ensures every determinant is a candidate key
 - FDs help by: Identifying all determinants and checking if they're proper keys



Database Design and Normalization

- Entity-Relationship (ER) Model
 - Conceptual representation of data entities and their relationships
- Normalization: 1NF, 2NF, 3NF, BCNF
- Denormalization: Trading off normalization for performance in certain scenarios
- Indexing: Improving query performance through data structure optimization
- Design Considerations: Balancing normalization, performance, and specific application needs



Features of Good Relational Designs

- Suppose we combine *instructor* and *department* into *in_dep*, which represents the natural join on the relations *instructor* and *department*

ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

- There is repetition of information
- Need to use null values (if we add a new department with no instructors)

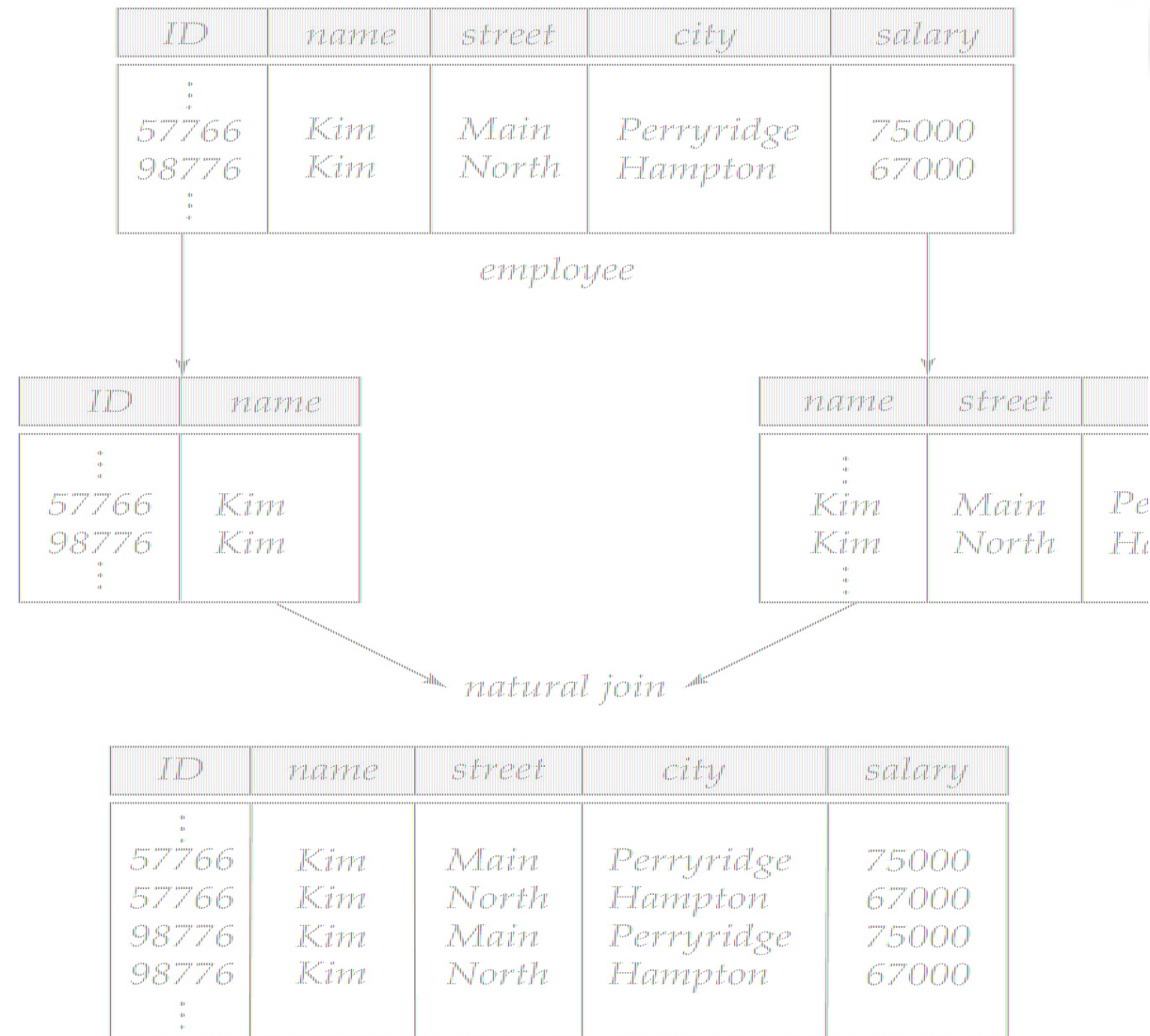


Decomposition

- The only way to avoid the repetition-of-information problem in the *in_dep* schema is to decompose it into two schemas – instructor and *department* schemas.
- Not all decompositions are good. Suppose we decompose
employee(*ID, name, street, city, salary*)
into
employee1 (*ID, name*)
employee2 (*name, street, city, salary*)
- This decomposition is **problematic because two employees could have the same name. If this happens, you would lose the ability to link the correct street and city to the correct employee ID.**
- This is called a **lossy decomposition** because you **cannot reconstruct the original employee table from these new, smaller tables.** The next slide will show how this loss of information happens.



A Lossy Decomposition



Lossless Decomposition

- A decomposition is considered **lossless** when you can perfectly reconstruct the original table by rejoining the decomposed tables
- This means that no information is lost and no extra, incorrect information is created
- This is the goal of proper database design.
- Formally, if a relation R is decomposed into relations R1 and R2, the decomposition is lossless if and only if the following equation is true
- Where:
$$\pi_{R_1}(r) \bowtie \pi_{R_2}(r) = r$$
 - r represents a specific instance of the relation R.
 - π (the Greek letter pi) is the **projection** operator, which selects specific columns from a relation.
 - $\bowtie$ (the "bowtie" symbol) is the **natural join** operator, which combines rows from two relations based on their common attributes.



Lossy Decomposition

- A decomposition is **lossy** when rejoining the decomposed tables results in a relation that is **not** identical to the original table
- This happens because the join operation introduces spurious, or incorrect, tuples that were not in the original data
- This is a sign of a bad decomposition that should be avoided.
- Formally, if a relation R is decomposed into relations R1 and R2, the decomposition is lossy if the following is true:

$$r \subset \pi_{R_1}(r) \bowtie \pi_{R_2}(r)$$

- This means that the result of the natural join is a **proper superset** of the original relation r, as it contains all the original tuples plus additional incorrect ones. The natural join operation should be a perfect reconstruction, not one that adds information that wasn't there to begin with.



Example of Lossless Decomposition

- Decomposition of $R = (A, B, C)$
 $R_1 = (A, B) R_2 = (B, C)$

A	B	C
α	1	A
β	2	B

r

A	B
α	1
β	2

$\Pi_{A,B}(r)$

B	C
1	A
2	B

$\Pi_{B,C}(r)$

$\Pi_A(r) \bowtie \Pi_B(r)$

A	B	C
α	1	A
β	2	B



Normalization Theory

- Decide whether a particular relation R is in “good” form.
- In the case that a relation R is not in “good” form, decompose it into set of relations $\{R_1, R_2, \dots, R_n\}$ such that
 - Each relation is in good form
 - The decomposition is a lossless decomposition
- Our theory is based on:
 - Functional dependencies
 - Multivalued dependencies



Functional Dependencies

- There are usually a variety of constraints (rules) on the data in the real world.
- For example, some of the constraints that are expected to hold in a university database are:
 - Students and instructors are uniquely identified by their ID.
 - Each student and instructor has only one name.
 - Each instructor and student is (primarily) associated with only one department.
 - Each department has only one value for its budget, and only one associated building.



Functional Dependencies (Cont.)

- An instance of a relation that satisfies all such real-world constraints is called a **legal instance** of the relation;
- A legal instance of a database is one where all the relation instances are legal instances
- Constraints on the set of legal relations.
- Require that the value for a certain set of attributes determines uniquely the value for another set of attributes.
- A functional dependency is a generalization of the notion of a *key*.



Functional Dependencies Definition

- Let R be a relation schema

$$\alpha \subseteq R \text{ and } \beta \subseteq R$$

- The **functional dependency**

$$\alpha \rightarrow \beta$$

holds on R if and only if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes α , they also agree on the attributes β . That is,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- Example: Consider $r(A,B)$ with the following instance of r .

1	4
1	5
3	7

- On this instance, $B \rightarrow A$ hold; $A \rightarrow B$ does **NOT** hold,



Closure of a Set of Functional Dependencies

- Given a set F set of functional dependencies, there are certain other functional dependencies that are logically implied by F .
 - If $A \rightarrow B$ and $B \rightarrow C$, then we can infer that $A \rightarrow C$
- The set of **all** functional dependencies logically implied by F is the **closure** of F .
- We denote the *closure* of F by F^+ .



Superkey and Candidate Key

- **Superkey:** A set of attributes K is a superkey for a relation schema R if and only if $K \rightarrow R$.
 - This means that the values of the attributes in K can uniquely identify every tuple in the relation.
 - K is a superkey for relation schema R if and only if $K \rightarrow R$
- **Candidate Key:** A candidate key is a minimal superkey
 - This means it uniquely identifies a tuple, and if you remove any attribute from the key, it is no longer a superkey
 - K is a candidate key for R if and only if
 - $K \rightarrow R$, and for no $\alpha \subset K$, $\alpha \rightarrow R$
-



Functional Dependencies

- Functional dependencies allow us to express constraints that cannot be expressed using superkeys. Consider the schema:

in_dep (ID, name, salary, dept_name, building, budget).

We expect these functional dependencies to hold:

dept_name → building

ID → building

but would not expect the following to hold:

name → salary

- The statement that we would "not expect this to hold" is correct. This is because a department contains multiple employees, and each employee can have a different salary.



Use of Functional Dependencies

Functional dependencies (FDs) are a fundamental concept in database design, used to ensure data integrity and prevent inconsistencies. Here's a clearer breakdown of their purpose:

What are Functional Dependencies Used For?

1. **To Define Constraints on Data:** FDs act as rules that specify what kind of data is considered valid or "legal" for a given table. They define the relationships between attributes and ensure that data is consistent and meaningful.
2. **To Test if Data is Valid:** Given a set of FDs, you can check any instance of a table to see if it violates these rules. If a table instance follows all the specified FDs, we say it **satisfies** them.
3. **To Ensure Rules Hold for all Future Data:** The goal is not just for the current data to be valid, but for the FDs to hold true for *all possible legal instances* of the table. We say that a set of FDs **holds** on a table schema if every valid instance of that table must satisfy those FDs.



Trivial Functional Dependencies

- A trivial functional dependency is a rule that is **always true**, no matter what data is in your table. It doesn't give you any new or useful information.
- The rule is $\alpha \rightarrow \beta$ is trivial if β is a subset of α .
- Let's break that down with your examples:
 - $ID, name \rightarrow ID$: The attributes on the right side (ID) are part of the attributes on the left side ($ID, name$). This rule is basically saying, "If you know a person's ID and $name$, can you figure out their ID ?" The answer is always yes, because you already have the ID on the left side! This is a trivial and useless rule.
 - $name \rightarrow name$: The right side ($name$) is a subset of the left side ($name$). This rule says, "If you know a person's $name$, can you figure out their $name$?" Again, the answer is always yes, because you already have the $name$.



Lossless Decomposition

- We can use functional dependencies to show when certain decomposition are lossless.
- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \prod_{R_1}(r) \bowtie \prod_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is lossless decomposition if at least one of the following dependencies is in F^+ :
 - $R_1 \cap R_2 \rightarrow R_1$
 - $R_1 \cap R_2 \rightarrow R_2$
- The above functional dependencies are a sufficient condition for lossless join decomposition; the dependencies are a necessary condition only if all constraints are functional dependencies



Example of Lossy Decomposition (1/2)

The Original Table: **R**

Imagine a single table called **R** that holds all the information about students, their courses, and their grades.

- **R** has the columns: (**StudentID**, **StudentName**, **CourseID**, **CourseName**, **Grade**)

The Rules (Functional Dependencies): **F**

The rules tell us what information can be uniquely determined by other information.

- **StudentID** $\rightarrow$ **StudentName**: If you know the **StudentID**, you know the **StudentName**.
- **CourseID** $\rightarrow$ **CourseName**: If you know the **CourseID**, you know the **CourseName**.
- **{StudentID, CourseID}** $\rightarrow$ **Grade**: To find a student's **Grade**, you need to know **both** their **StudentID** and the **CourseID** they took.



Example of Lossy Decomposition (2/2)

- **The "Decomposition"**

- The goal of decomposition is to split a large table into smaller, more efficient ones without losing information
- They tried to split **R** into two smaller tables:
 - **R1 = (StudentID, StudentName, CourseID)**: This table contains student and course information.
 - **R2 = (CourseID, CourseName, Grade)**: This table contains course and grade information.



Example of Lossy Decomposition (2/2)

- **Why This Decomposition is "Lossy"**
 - A decomposition is "lossy" when you **cannot perfectly rebuild the original table** by joining the smaller tables back together. Some critical information is lost.
 - In this case, the decomposition is lossy because the two new tables **do not share a common key** that links all the original information.
 - R1 has the key {StudentID, CourseID}.
 - R2 has the key CourseID
- The original relationship between a student's ID and their Grade is determined by the combination of {StudentID, CourseID}
- When you split the table, you put StudentID in R1 and Grade in R2.
- **The problem is that there is no key linking StudentID to Grade across the two tables.** R1 and R2 only have CourseID in common.



Example of Lossless Decomposition

- $R = (\text{StudentID}, \text{Name}, \text{CourseID}, \text{CourseName}, \text{Grade})$
- $F = \{\text{StudentID} \rightarrow \text{Name}, \text{CourseID} \rightarrow \text{CourseName}, \{\text{StudentID}, \text{CourseID}\} \rightarrow \text{Grade}\}$
- Decomposition:
 - $R1 = (\text{StudentID}, \text{Name})$
 - $R2 = (\text{CourseID}, \text{CourseName})$
 - $R3 = (\text{StudentID}, \text{CourseID}, \text{Grade})$
- This is a lossless decomposition because:
 - $R1 \cap R3 = \{\text{StudentID}\}$ and $\text{StudentID} \rightarrow \text{Name}$ (in $R1$)
 - $R2 \cap R3 = \{\text{CourseID}\}$ and $\text{CourseID} \rightarrow \text{CourseName}$ (in $R2$)



Dependency Preservation

- A decomposition is called "**dependency preserving**" if all the original rules (functional dependencies) can be checked by looking at just the new, smaller tables.
- In other words, no rule is split across tables in a way that forces you to do a slow and costly join to check it.
- If you have to do a slow operation like a Cartesian Product to check a rule, the decomposition is **NOT dependency preserving**.



Simple Analogy (1/2)

- **Original Big Basket**
 - You have one big basket with all your clothes: shirts, socks, and pants.
- **Original Rules**
 - Your rule is: "Every sock must have a matching pair."
- **Easy Check**
 - To check the rule, you just look inside the single basket and match up the socks. This is fast and easy.
- **Decomposition (Splitting the Laundry)**
 - You decide to put your shirts in a closet, your pants in a dresser, and your socks in a sock drawer. This seems more organized.



Simple Analogy (2/2)

- **The Problem**
 - Now, to check your rule ("Every sock must have a matching pair"), you have to collect all the socks from all the different places and put them back together. This is a hassle.
- **"Dependency Preserving" Decomposition**
 - This is like if you only had one rule about socks, and you put all the socks together in one drawer. All the information you need to check that one rule is right there, in one place. The decomposition is "rule-friendly."
- **"Not Dependency Preserving" Decomposition**
 - This is like if you split the socks up into different drawers. Now, checking the rule requires extra effort. This is not a "rule-friendly" decomposition.



Example of Non-dependency Preserving Decomposition (1/4)

- Imagine a big table **R** with all the information about students, courses, instructors, and departments.
- The rules (**F**) are:
 - $\{\text{StudentID}, \text{CourseID}\} \rightarrow \text{Grade}$: To find a grade, you need to know the student and the course.
 - $\text{InstructorID} \rightarrow \text{InstructorName}$: An instructor's ID tells you their name.
 - $\text{CourseID} \rightarrow \text{InstructorID}$: A course ID tells you who teaches it.
 - $\text{InstructorID} \rightarrow \text{DepartmentID}$: An instructor's ID tells you their department.
- **Hidden Rule**: There's a rule that's not explicitly listed but is logically true. Because a **CourseID** tells you the **InstructorID** ($\text{CourseID} \rightarrow \text{InstructorID}$) and the **InstructorID** tells you the **DepartmentID** ($\text{InstructorID} \rightarrow \text{DepartmentID}$), it's also true that a **CourseID** tells you the **DepartmentID**.
 - **Derived Rule**: $\text{CourseID} \rightarrow \text{DepartmentID}$



Example of Non-dependency Preserving Decomposition (2/4)

The Decomposition (Splitting the Table)

The big table **R** is split into four smaller tables:

- **R1 = (StudentID, CourseID, Grade)**: This table handles student grades.
- **R2 = (InstructorID, InstructorName)**: This table handles instructor names.
- **R3 = (CourseID, InstructorID)**: This table handles which instructor teaches which course.
- **R4 = (InstructorID, DepartmentID)**: This table handles which department an instructor belongs to.



Example of Non-dependency Preserving Decomposition

(3/4)

- The goal is to check all the original rules by only looking at the smaller tables
 - $\{StudentID, CourseID\} \rightarrow Grade?$ **Yes**, we can check this rule just by looking at R1.
 - $InstructorID \rightarrow InstructorName?$ **Yes**, we can check this rule just by looking at R2.
 - $CourseID \rightarrow InstructorID?$ **Yes**, we can check this rule just by looking at R3.
 - $InstructorID \rightarrow DepartmentID?$ **Yes**, we can check this rule just by looking at R4.
- So far, so good. But what about the **derived rule**?
 - $CourseID \rightarrow DepartmentID?$ **No!**
 - i. $CourseID$ is in table R3.
 - ii. $DepartmentID$ is in table R4.
 - There is no single table that contains both $CourseID$ and $DepartmentID$.



Example of Non-dependency Preserving Decomposition (4/4)

- To check this rule, you would have to:
 1. Take the **CourseID** from **R3** to find the **InstructorID**.
 2. Then, take that **InstructorID** and look it up in **R4** to find the **DepartmentID**.
- In a database, this is done by a **join** operation, where you temporarily combine tables **R3** and **R4** based on their common column (**InstructorID**)
 - This is a **computationally expensive** process
 - It's much slower than just checking a rule within a single table.
- Because one of the original rules (the derived one) cannot be checked efficiently on a single, new table, the decomposition is **not dependency preserving**.
 - It makes the database harder and slower to maintain because you can't easily perform all the necessary data quality checks.



Example of Dependency Preserving Decomposition (1/3)

- The goal of "dependency preservation" is to split a large table into smaller ones in a way that allows you to **easily check all your original data rules without doing any complicated work.**
- The key is that for every single rule, all the information needed to check it must be found within **one single, smaller table.**
- The database is split into four smaller, more focused tables:
 - **R1 = (StudentID, CourseID, Grade):** This table handles student grades.
 - **R2 = (InstructorID, InstructorName, DepartmentID):** This table handles instructor information.
 - **R3 = (CourseID, InstructorID):** This table connects courses to instructors.
 - **R4 = (CourseID, DepartmentID):** This table connects courses to departments.



Example of Dependency Preserving Decomposition (2/3)

Why It's "Dependency Preserving" (The Solution)

- The decomposition is "dependency preserving" because we can check **every single rule** just by looking at a single table. No complex combining of tables is needed.
- Let's check each rule:
 - **Rule 1: {StudentID, CourseID} → Grade**
 - **Can we check this easily?** Yes. All the information needed (StudentID, CourseID, Grade) is in table R1. You can quickly scan R1 to make sure the rule is not broken.
 - **Rule 2: InstructorID → InstructorName**
 - **Can we check this easily?** Yes. Both InstructorID and InstructorName are in table R2.
 - **Rule 3: CourseID → InstructorID**
 - **Can we check this easily?** Yes. Both CourseID and InstructorID are in table R3.



Example of Dependency Preserving Decomposition (2/4)

- **Rule 4: $\text{InstructorID} \rightarrow \text{DepartmentID}$**
 - **Can we check this easily?** Yes. Both InstructorID and DepartmentID are in table $R2$.
- **Rule 5 (the derived rule): $\text{CourseID} \rightarrow \text{DepartmentID}$**
 - **Can we check this easily?** Yes! This is the most important part. Even though this rule was derived from a combination of other rules, the new table $R4$ was specifically created to hold this information.
 - This means you can check this rule without having to join two different tables together.
- Because every single rule—whether explicitly stated or derived—can be checked within a single, new table, this decomposition is **dependency preserving**
- It makes the database easy and fast to maintain and keep clean.



Conclusions (1/2)

- Relational database design is crucial for creating efficient, consistent, and maintainable databases
- Key concepts covered:
 - Functional dependencies and their role in database design
 - Lossless decomposition vs. lossy decomposition
 - Dependency preservation in database decomposition
- Good relational design aims to:
 - Minimize data redundancy
 - Ensure data integrity
 - Facilitate efficient data access and updates
 - Support scalability and flexibility



Conclusions (2/2)

- Normalization, guided by functional dependencies, helps achieve these goals by:
 - Eliminating anomalies (insertion, update, deletion)
 - Organizing data into well-structured relations
- When decomposing relations, it's important to consider both:
 - Lossless decomposition: to preserve all original information
 - Dependency preservation: to efficiently enforce constraints
- Balancing these factors leads to robust database designs that support data integrity, performance, and maintainability.



References

- Avi Silberschatz, Henry F. Korth, S. Sudarshan. Database System Concepts [Internet]. New Haven: Yale University; [cited 2024 Sep 10]. Available from: <https://codex.cs.yale.edu/avi/db-book/slides-dir/index.html>
- Anthropic. Claude AI [Internet]. San Francisco: Anthropic; 2024 [cited 2024 Sep 10]. Available from: <https://www.anthropic.com>

